

Assignment-09(Lab-9.5)

Course : AI Assisted Coding

Topic:- Documentation Generation: Automatic Documentation and Code Comments

Student Details:

Name : Batti Chandan Singh

Hall Ticket No : 2303A51515

Batch : 22

Task-01

Final Optimal Prompt

Generate a short and concise functional summary (docstring) for each function in this script. Do not explain implementation details, only describe the purpose of each function.

Code Screenshot

```
Assignments python file > Assignment-09(Lab-9.2).py > main
1  #---Lab-9.5-Task-01/05---
2  """Generate a short and concise functional summary (docstring) for each function in this script.
3  Do not explain implementation details, only describe the purpose of each function. no need one line description.
4  def calculate_area(radius):
5      """Calculate the area of a circle given its radius.
6
7      Args:
8          radius (float): The radius of the circle.
9
10     Returns:
11         float: The area of the circle.
12     """
13     import math
14     return math.pi * radius ** 2
15 def calculate_circumference(radius):
16     """Calculate the circumference of a circle given its radius.
17
18     Args:
19         radius (float): The radius of the circle.
20
21     Returns:
22         float: The circumference of the circle.
23     """
24     import math
25     return 2 * math.pi * radius
26 def calculate_diameter(radius):
27     """Calculate the diameter of a circle given its radius.
28
29     Args:
30         radius (float): The radius of the circle.
31
32     Returns:
33         float: The diameter of the circle.
34     """
35     return 2 * radius
```

```

    return 2 * Radius

def main():
    """Main function to execute the program.

    This function prompts the user for the radius of a circle, calculates the area, circumference, and diameter,
    and then prints the results to the console.
    """
    radius = float(input("Enter the radius of the circle: "))
    area = calculate_area(radius)
    circumference = calculate_circumference(radius)
    diameter = calculate_diameter(radius)

    print(f"Area: {area:.2f}")
    print(f"Circumference: {circumference:.2f}")
    print(f"Diameter: {diameter:.2f}")

```

Output Screenshot

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\python\file/Assignment-09(Lab-9.2).py"
● PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\python\file/Assignment-09(Lab-9.2).py"
Enter the radius of the circle: 6
Area: 113.10
Circumference: 37.70
Diameter: 12.00
○ PS D:\6th Sem 2025-26\AI-Assisted Coding>

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task demonstrates how AI can add concise documentation to Python functions. The prompt instructs the model to generate brief, high-level summaries without discussing the internal code logic. Each function receives a docstring that clearly describes its purpose, making the code easier to understand and maintain. The goal is to enhance readability while keeping the explanation minimal and technically accurate. The AI-generated comments describe what each function does, not how it works, which aligns with good documentation practices. Such summaries help future developers quickly understand the function's role in the program.

Task-02

Final Optimal Prompt

Add comments explaining only the decision-making logic and loop behavior in this code.

Do not explain syntax or basic Python concepts."

Code Screenshot

This file performs basic mathematical operations and prints results for demonstration purposes. It includes functions for `is_prime()` along with statements that display the computed output.

Code Screenshot

```
1 #---Lab-9.5-Task-03/05---
2 """Add comments explaining only the decision-making logic and loop behavior in this code.
3 Do not explain syntax or basic Python concepts."""
4
5 """
6 This file performs basic mathematical operations and prints results
7 for demonstration purposes. It includes functions for is_prime() along with statements that display the computed output
8 """
9
10 def is_prime(num):
11     if num <= 1:
12         return False
13     for i in range(2, int(num**0.5) + 1):
14         if num % i == 0:
15             return False
16     return True
17
18 number = int(input("Enter a number to check if it is prime: "))
19 if is_prime(number):
20     print(f"{number} is a prime number.")
21 else:
22     print(f"{number} is not a prime number.")
23
24 #Example use test cases:
25 numbers_to_test = [1, 2, 3, 4, 5, 10, 11, 12, 13, 14]
26 for num in numbers_to_test:
27     if is_prime(num):
28         print(f"{num} is a prime number.")
29     else:
30         print(f"{num} is not a prime number.")
```

Output Screenshot

```
● PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\python
file/Assignment-09(Lab-9.2).py"
Enter a number to check if it is prime: 435
435 is not a prime number.
1 is not a prime number.
2 is a prime number.
3 is a prime number.
4 is not a prime number.
5 is a prime number.
10 is not a prime number.
11 is a prime number.
12 is not a prime number.
13 is a prime number.
14 is not a prime number.
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task adds a file-level overview that provides a summarized description of the Python program. The prompt asks AI to explain the overall purpose without focusing on individual lines. The generated overview explains that the file demonstrates basic arithmetic operations using simple functions. Such top-level documentation is helpful for developers reviewing the file because it tells them what to expect before reading the code. It contributes to better project organization, readability, and maintainability by offering

immediate context. This small but essential addition increases clarity, especially in larger projects with multiple files.

Task-04

Final Optimal Prompt

Rewrite the existing comments to make them clearer, more consistent, and easier to understand without changing the technical meaning

Code Screenshot

```
112 #---Lab-9.6-Task-04/05---
113 """Rewrite the existing comments to make them clearer, more consistent, and easier to understand without
114 changing the technical meaning"""
115 def calculate_fibonacci(n):
116     """calculate the Fibonacci sequence up to the nth number.
117
118     Args:
119         n (int): The number of Fibonacci numbers to generate.
120
121     Returns:
122         list: A list containing the Fibonacci sequence up to the nth number.
123     """
124     if n <= 0:
125         return []
126     elif n == 1:
127         return [0]
128     elif n == 2:
129         return [0, 1]
130     else:
131         fibonacci_sequence = [0, 1]
132         # Loop to generate Fibonacci numbers until we reach n
133         for i in range(2, n):
134             next_fib = fibonacci_sequence[i - 1] + fibonacci_sequence[i - 2]
135             fibonacci_sequence.append(next_fib)
136         return fibonacci_sequence
137 # Example usage:
138 n = int(input("Enter the number of Fibonacci numbers to generate: "))
139 fibonacci_numbers = calculate_fibonacci(n)
140 print(f"Fibonacci sequence up to the {n}th number: {fibonacci_numbers}")
141
142
```

Output Screenshot

```
PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\python 3.13.6\python.exe" "d:/6th Sem 2025-26/AI-Assisted Coding/Assignments python/Assignment-09(Lab-9.2).py"
Enter the number of Fibonacci numbers to generate: 45
Fibonacci sequence up to the 45th number: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 67
10946, 17711, 28657, 46368, 75025, 121393, 196418, 317811, 514229, 832040, 1346269, 2178309, 3524578, 5702887, 9227465, 14930352,
57817, 39088169, 63245986, 102334155, 165580141, 267914296, 433494437, 701408733]
PS D:\6th Sem 2025-26\AI-Assisted Coding>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

In this task, AI improves vague or inconsistent comments in the original function. The rewritten documentation is clearer, more professional, and easier to understand. The

function now includes a concise docstring describing its purpose, and the inline comment has been rewritten to reflect what the operation does. This helps maintain consistent tone and clarity in documentation without altering the code's behavior. Such refinement is essential for readable and maintainable codebases. AI assists by ensuring documentation is both accurate and high-quality.

Task-05

Final Optimal Prompt

Write documentation for this function, including a description of its purpose, its parameters, and its return value.

Code Screenshot

```
###Lab-9.6-Task-05/05###
"""Write documentation for this function, including a description of its purpose, its parameters, and its return value."""
def calculate_gcd(a, b):
    """calculate the greatest common divisor (GCD) of two numbers using the Euclidean algorithm.

    Args:
        a (int): The first number.
        b (int): The second number.

    Returns:
        int: The greatest common divisor of the two numbers.
    """
    while b:
        a, b = b, a % b
    return abs(a)
# Example usage:
num1 = int(input("Enter the first number: "))
num2 = int(input("Enter the second number: "))
gcd_result = calculate_gcd(num1, num2)
print(f"The greatest common divisor of {num1} and {num2} is: {gcd_result}")
```

Output Screenshot

```
...
● PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\python 3.13
  ile/Assignment-09(Lab-9.2).py"
  Enter the first number: 4
  Enter the second number: 5
  The greatest common divisor of 4 and 5 is: 1
○ PS D:\6th Sem 2025-26\AI-Assisted Coding> █
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

Task 5 analyzes how prompt detail affects AI-generated documentation quality. The brief prompt produces a short, incomplete docstring that lacks clarity and provides little guidance. In contrast, the detailed prompt results in a comprehensive, professional-level docstring containing parameter descriptions, expected behavior, and error conditions. This demonstrates that well-crafted prompts significantly improve AI output quality. Developers can leverage this insight to generate more useful documentation by giving specific instructions.